IFDP Instructions for Lovable & AI Assistants

How to use the Investigation-First Development Protocol with AI coding assistants like Lovable, Cursor, Claude, etc.

Overview

When working with AI assistants on complex production issues, structure your requests around IFDP phases. This ensures:

1. **Rigorous root cause analysis** before implementing fixes
2. **Clear approval points** before proceeding to implementation
3. **Observable, auditable changes** with comprehensive logging
4. **Measurable verification** of fix effectiveness

Phase A: Investigation with AI Assistants

What to Request

Phase A Goal: Gather evidence, not write code.

```
I need help with Phase 1A investigation for [issue description].

Here's what I know:
- [Symptom or error message]
- [When it started or how often it occurs]
- [Relevant systems or files]

Please help me:
1. Map the complete logic flow with file paths and line numbers
2. Run database/log queries to gather evidence
3. Identify 3-5 potential root causes ranked by likelihood
4. Explain what data supports each hypothesis

DO NOT write code yet. Just gather evidence.
```

What the Assistant Should Deliver

1. **Flow Diagram** - Logic flow with file paths and line numbers
2. **Evidence Summary** - Query results, metrics, logs
3. **Ranked Hypotheses** - Root causes with supporting data
4. **Risk Assessment** - Which fixes are lowest-risk
5. **Approval Point** - Clear stopping point before Phase B

Example Phase A Request

```
I need help with Phase 2A investigation for a rendering bug.

Symptoms:
- Gainesville videos look repetitive
- Other cities have more variety
- Started after the last Creatomate API update

Please help me:
1. Map the video rendering flow in _shared/video-render.ts
2. Check what happens when Creatomate returns "No Template"
3. Query the logs to see how often we're hitting API errors
4. Compare rendering diversity metrics before/after the Creatomate update
5. Form a hypothesis about why fallback is happening

Return the analysis but no code changes please.
```

Phase B: Implementation with AI Assistants

Prerequisites

- [ ] Phase A findings are documented and approved
- [ ] Root cause hypothesis is clear
- [ ] Specific fix strategy is chosen from Phase A options
- [ ] You've reviewed the Phase A findings

What to Request

Phase B Goal: Implement approved fix + add observability.

```
Phase 1A investigation is complete. Root cause: [finding from Phase A].

Approved fix strategy: [specific option chosen]

Now for Phase 1B implementation, please:

1. Implement the fix in [file path]
   - Change [current behavior] to [new behavior]
   - Keep changes minimal and surgical

2. Add observability logging to [specific locations]
   - Log format: { "event_type": "fix_checkpoint", ... }
   - Include context: city, slot, provider
   - Capture: success/failure, status codes, error messages

3. Create unit tests covering:
   - [Happy path test]
   - [Error handling test]
   - [Edge case test]

4. Run tests to verify 100% pass rate

Return code only—I'll handle deployment and Phase C monitoring.
```

Logging Template for AI Assistants

Provide this template to the assistant:

```
// Logging template for Phase B implementation
const logCheckpoint = (eventType, context, details) => {
  return {
    event_type: eventType,
    timestamp: new Date().toISOString(),
    context: {
      city: context.city || "unknown",
      slot: context.slot || "unknown",
      provider: context.provider || "unknown",
      user_id: context.user_id || "anonymous"
    },
    message: details.message,
    error_details: {
      http_status: details.http_status || null,
      error_message: details.error_message || null,
      response_body_excerpt: details.response_excerpt || null
    }
  };
};

// Example usage:
console.log(logCheckpoint("fix_checkpoint",
  { city: "Miami", slot: "afternoon", provider: "Creatomate" },
  {
    message: "API call succeeded with expected response",
    http_status: 200,
    error_message: null
  }
));
```

Example Phase B Request

```
Phase 2A investigation revealed: Silent API failure in Creatomate is causing 100%
fallback to repetitive rendering style.

Approved fix: Add retry logic with exponential backoff + structured logging.

Phase 2B implementation:

1. In _shared/video-render.ts, modify the Creatomate API call:
   - Implement retry logic (3 attempts, exponential backoff)
   - Log each attempt with status and error details
   - Fall back gracefully if all retries fail

2. Add structured logging at these checkpoints:
   - Before API call (which template is being used)
   - On successful response (template used, response time)
   - On failure (HTTP status, error message)

3. Create tests:
   - Mock successful API response → verify correct rendering
   - Mock API failure → verify retry logic triggers
   - Mock all retries fail → verify graceful fallback

4. Run tests to ensure 100% pass.

Return the implementation code and test file.
```

Phase C: Verification Metrics

What to Request

Phase C Goal: Prove the fix works in production.

```
Phase 1B implementation deployed. Now for Phase 1C verification:

I'll monitor production for 24-48 hours.
At the end, I'll need you to help me:

1. Compare these metrics BEFORE vs AFTER the fix:
   - [Metric 1, e.g., "rendering diversity %"]
   - [Metric 2, e.g., "API error rate"]
   - [Metric 3, e.g., "fallback frequency"]

2. Run these queries:
   - [Query to get before metrics]
   - [Query to get after metrics]

3. Create a comparison table showing:
   - Metric name
   - Before value
   - After value
   - % improvement
   - Status (improved/degraded/no change)

4. Check for regressions in related areas:
   - [System A: expected impact]
   - [System B: expected impact]

I'll gather the data; you help me organize and interpret it.
```

Example Phase C Request

```
Phase 2B deployment complete. Monitoring production for 48 hours.

After 48 hours, I have these logs and metrics:

[Paste query results here]

Help me:
1. Create a table comparing rendering diversity BEFORE/AFTER
2. Show API error rate reduction
3. Calculate % improvement in each metric
4. Check: did we introduce any new errors?
5. Confirm: are observability logs showing expected patterns?

Final verdict: should we mark this as GO or NO-GO?
```

Complete IFDP Workflow with AI Assistants

Step 1: Phase A Investigation (2–4 hours)

```
Request: "Help with Phase 1A investigation for [issue]"
Assistant: Returns evidence, hypotheses, ranked fixes
You: Review findings, choose approved fix strategy
Approval Point: "Proceed to Phase 1B implementation"
```

Step 2: Phase B Implementation (2–8 hours)

```
Request: "Implement Phase 1B based on Phase 1A findings"
Assistant: Returns code, tests, observability logging
You: Review code, run tests locally, deploy
Approval Point: "Phase 1B deployed, ready for Phase 1C"
```

Step 3: Phase C Verification (24–48 hours)

```
You: Monitor production
Request: "Help me verify Phase 1C metrics"
Assistant: Analyzes data, creates comparison, recommends GO/NO-GO
You: Final decision and sign-off
Completion: "Phase 1C verification complete. Final verdict: GO"
```

Best Practices for AI Assistants

Do’s ✓

- **Be explicit about phase.** Use “Phase 1A”, “Phase 1B”, “Phase 1C” in requests
- **Provide context.** Share Phase A findings before requesting Phase B code
- **Ask for specific outputs.** “Return a comparison table” is better than “compare the metrics”
- **Include examples.** Show the expected format for logs, tests, or metrics
- **Request step-by-step.** Break Phase B into smaller numbered tasks
- **Verify completeness.** Ask the assistant to confirm all logging and tests are included

Don’ts ✗

- **Don’t skip Phase A.** Don’t jump to code without evidence
- **Don’t merge phases.** Don’t ask for “Phase A + B in one request”
- **Don’t vague requests.** “Fix the bug” doesn’t work; specify the hypothesis and approach
- **Don’t ignore logging.** Every Phase B must include structured logging
- **Don’t skip testing.** Every Phase B must include unit tests with 100% pass rate
- **Don’t assume GO.** Always run Phase C verification and compare metrics

---

Prompt Templates (Copy & Paste)

Phase A Investigation Template

```
I need help with Phase [X]A investigation for: [issue name/description]

Context:
- Symptom: [what's broken]
- Timeline: [when did it start]
- Affected systems: [which services/files]

Please help me:
1. [Specific analysis request 1]
2. [Specific analysis request 2]
3. [Specific analysis request 3]
4. Rank potential root causes by likelihood
5. Return a summary with approval point

DO NOT write code. Evidence gathering only.
```

Phase B Implementation Template

```
Phase [X]A investigation complete. Root cause: [finding]

Approved fix strategy: [chosen option]

For Phase [X]B implementation:

1. Implement the fix in [file]:
   - [specific change 1]
   - [specific change 2]

2. Add observability logging at [locations]:
   - Include: event_type, context, error_details
   - Use the logging template provided below

3. Create tests for:
   - [test case 1]
   - [test case 2]
   - [test case 3]

4. Ensure 100% test pass rate

Logging template: [provide template]

Return: Code file + test file only. No deployment—I'll handle that.
```

Phase C Verification Template

```
Phase [X]B deployed successfully.

For Phase [X]C verification:

I've monitored production for 48 hours. Here's the data: [paste data]

Please help me:
1. Create a before/after comparison table for these metrics:
   - [metric 1]
   - [metric 2]
   - [metric 3]

2. Calculate % improvement for each metric

3. Check for regressions in:
   - [system 1]
   - [system 2]

4. Summarize: Is this fix working? Any concerns?

5. Recommend: GO or NO-GO?
```

Troubleshooting

Issue: Assistant writes code during Phase A

Solution: Explicitly state “DO NOT write code” and “Evidence gathering only.” Redirect to analysis tasks.

Issue: Phase B code has no logging

Solution: Always provide the logging template. After receiving code, ask: “Are observability checkpoints included at [locations]?”

Issue: Tests are incomplete

Solution: Request specific test cases by name. Ask: “Create tests for: happy path, error case, edge case. Return 100% pass results.”

Issue: Unclear if fix is working (Phase C)

Solution: Ask the assistant to compare metrics side-by-side in a table format. Include “before”, “after”, and “% change” columns.

Integration with GitHub Commits

Use IFDP phase numbers in commit messages for clarity:

```
# Phase A: Investigation
git commit -m "Phase 5A investigation: root cause - hardcoded banned fragments"

# Phase B: Implementation
git commit -m "Phase 5B implementation: rewrite 6 templates, add observability log-
ging"

# Phase C: Verification
git commit -m "Phase 5C verification: GO - block rate improved 25% → 2%"
```

Reference

- **Full methodology:** See `IFDP_METHODOLOGY.md`
- **Quick reference:** See `IFDP_QUICK_REFERENCE.md`
- **Questions?** Review the FAQ section in `IFDP_METHODOLOGY.md`